

ECE 154A Lab 1 Report

Shravya Salem Sathish, David Uzunov

1 Hours Spent

We spent a total of 8 hours on this lab.

2 mult.s Assembly Code

Completed mult.s code pasted below with C equivalents in the comments:

```
add a2, zero, zero                # a2 = 0
LOOP:
    beq a1,zero,DONE              # while(a1!=0){
    andi t0,a1,1                  # t0 = a1 & 1;
    beq t0,zero,SHIFT             # if(t0!=0){
    add a2,a2,a0                  # a2 += a0;}
    SHIFT:
        srli a1,a1,1              # a1 = a1 >> 1;
        slli a0,a0,1              # a0 = a2 << 1;
    j LOOP                        # } // close out the while loop
DONE:
```

3 div.s Assembly Code

Completed div.s code pasted below with C equivalents in the comments:

```
# Set initial values.
# The remainder is initially
# equal to the dividend.
add a3, zero, a0                  # a3 = a0;

# If the divisor is zero,
# set the quotient to 0xFF
```

```

# and proceed to done.
bne a1, zero, divisorNonzero          # if (a1 != 0) { divisorNonZero; }
addi a2, zero, 0x000000FF             # a2 = 0x000000FF;
j done

divisorNonzero:
# For nonzero divisors,
# set the quotient to zero.
add a2, zero, zero                    # a2 = 0;

# If the dividend is lesser
# than the divisor, proceed to done.
bltu a0, a1, done                     # if (a0 < a1) { done; }

# Dividend and divisor are valid,
# so calculate their bit lengths.
# Initially store the dividend in t0
# and the divisor in t1. Store their current
# lengths in t2 and t3, respectively.
add t0, zero, a0                      # t0 = a0;
add t1, zero, a1                      # t1 = a1;
add t2, zero, zero                    # t2 = 0;
add t3, zero, zero                    # t3 = 0;

# Right shift t0 and increment t2
# until t0 is zero for the dividend
# bit length.
whileDividendNonzero:
beq t0, zero, whileDivisorNonzero     # while (t0 > 0)
srli t0, t0, 1                         # t0 >>= 1;
addi t2, t2, 1                         # t2 += 1;
j whileDividendNonzero

# Right shift t1 and increment t3
# until t1 is zero for the divisor
# bit length.
whileDivisorNonzero:
beq t1, zero, lenDone                  # while (t1 > 0)
srli t1, t1, 1                         # t1 >>= 1;
addi t3, t3, 1                         # t3 += 1;

```

```

j whileDivisorNonzero

lenDone:
# Left shift the divisor by the
# difference in bit length between the
# dividend and the divisor.
sub t4, t2, t3                # t4 = t2 - t3
sll a1, a1, t4                # a1 <= t4;

# Set up a for loop with t4 + 1
# iterations for the division.
addi t4, t4, 1                # t4 += 1;
add t5, zero, zero            # t5 = 0;

for:                            # for (; t5 < t4; t5++)
bge t5, t4, done

# If the remainder is greater than
# or equal to the divisor, subtract
# the divisor from the remainder,
# left shift the quotient by 1, and
# set the quotient LSB to 1.
bltu a3, a1, else              # if (a3 >= a1) { ... }
sub a3, a3, a1                 # a3 -= a1;
slli a2, a2, 1                 # a2 <= 1;
ori a2, a2, 1                  # a2 |= 1;
j skip

# Else just left shift the quotient by 1,
# setting the LSB to 0.
else:                          # else { a2 <= 1; }
slli a2, a2, 1

# Right shift the divisor by 1
# and increment the for loop counter.
skip:
srli a1, a1, 1                 # a1 >>= 1;
addi t5, t5, 1                 # t5 += 1;
j for

```

done:

End of division.